ACADEMIC MINI PROJECT PRESENTATION

GATHERLY : EVENT & TICKET MANAGEMENT

A Full-Stack Event Discovery, Booking & Automated Ticketing Platform Built with Next.js 16, PostgreSQL & Stripe

---

4 TEAM MEMBERS & MODULAR ROLE DISTRIBUTION (4 SLIDES PER MEMBER):

Member 1: Full-Stack Architect & Backend Lead [Member 1]

Key Modules: Next.js 16 App Router, JWT Authentication, REST APIs, Security Guards

Member 2: Frontend Lead & UI/UX Developer [Member 2]

Key Modules: React 19, Tailwind CSS v4, Multi-Category Discovery, Organizer Dashboard

Member 3: Database & ORM Engineer [Member 3]

Key Modules: PostgreSQL Relational Schema, Drizzle ORM, Concurrency & Inventory Control, DB Seeding

Member 4: Payment, Ticketing & DevOps Engineer [Member 4]

Key Modules: Stripe Checkout API, PDFKit E-Tickets, QR Codes, Testing & Vercel Deployment

Slide 1 (M1): System Architecture & Tech Stack

Full-Stack Architecture Overview

• **Next.js 16 Framework:** App Router architecture with React 19 providing a unified full-stack codebase.

• **Server & Client Boundary:** Server components for fast data fetching and client components for interactive UI forms.

• **Colocated API Routes:** Backend REST API routes organized cleanly under src/app/api/, eliminating CORS issues.

• **Turbopack Compiler:** Ultra-fast build and bundling tool ensuring quick hot-reloads and optimized bundles.

Directory Layout & Request Pipeline

• **Request Flow:** Browser Request -> Next.js Route Handler -> Auth Guard -> Drizzle ORM -> PostgreSQL.

• **Modular Structure:** Clean separation between src/app/ (routes), src/db/ (models), and src/lib/ (utilities).

• **State Sync:** Zustand client store synchronizes user authentication state via /api/auth/me.

• **Edge Ready:** Fully optimized for serverless edge deployment with low memory consumption.

Slide 2 (M1): Authentication & Role-Based Access Control

User Authentication Pipeline

- **Role-Based Access (RBAC):** Strict database separation between "organizer" (event host) and "attendee" (ticket buyer).
- **Password Encryption:** Passwords securely hashed using bcryptjs with 10 salt rounds before database storage.
- **Stateless JWT Tokens:** Tamper-proof tokens signed with private secret key containing user ID, email, and role.
- **Environment Security:** JWT_SECRET and credentials stored safely in environment variables, never in source code.

Session & Cookie Security Strategy

- **httpOnly Cookie Storage:** JWT tokens stored in httpOnly cookies, completely blocking Cross-Site Scripting (XSS) access.
- **SameSite Flag Protection:** Configured with SameSite: Lax to safeguard against Cross-Site Request Forgery (CSRF).
- **7-Day Session Validity:** Sessions auto-expire after 7 days; logout immediately clears the cookie header.
- **Auth Guard Helper:** src/lib/auth.ts decodes and verifies session tokens on all protected routes.

Slide 3 (M1): API Route Design & Server-Side Security

RESTful API Endpoint Structure

- **Authentication APIs:** /api/auth/register, /api/auth/login, /api/auth/logout, and /api/auth/me.
- **Events Catalog APIs:** GET /api/events for category & keyword filtering; POST /api/events to create shows.
- **Event Management APIs:** GET, PUT, and DELETE /api/events/[id] for managing specific event details.
- **Ticketing APIs:** /api/bookings for user history and /api/checkout for Stripe session generation.

Route Authorization & Error Handling

- **Ownership Verification:** Verifies event.organizerId === user.id before allowing update or delete operations.
- **Role Permission Guards:** Blocks attendees from accessing organizer management endpoints with HTTP 403 Forbidden.
- **Consistent HTTP Codes:** Standard status codes: 400 Bad Request, 401 Unauthorized, 404 Not Found, 500 Server Error.
- **Input Validation:** Validates that show dates are in the future and ticket prices are valid positive numbers.

Slide 4 (M1): Challenges Overcome & Deliverables

Technical Challenges Solved

• **Challenge: State Hydration:** Synchronizing client-side Zustand store with server httpOnly session cookies.

• **Solution Implemented:** Built /api/auth/me route to safely verify and bootstrap user session on layout mount.

• **Challenge: Cookie Headers:** Handling asynchronous cookie mutation headers in Next.js 16 route handlers.

• **Solution Implemented:** Leveraged Next.js cookies() API to set and clear serialized response headers safely.

Key Architectural Deliverables

• **Modular Controller Design:** Clean, isolated API route handlers with reusable database and authentication helpers.

• **Defense in Depth:** Validation enforced on both client forms and server routes for absolute security.

• **Stateless Scalability:** JWT architecture avoids heavy session tables, enabling fast serverless execution.

• **Team Coordination:** Established clean JSON response formats so frontend and database could work in parallel.

Slide 1 (M2): Design System & User Interface Architecture

UI Design System & Aesthetics

- **Ticketing Platform Theme:** Clean, modern visual design inspired by BookMyShow and Eventbrite.
- **Curated Dark Palette:** Slate canvas (#0F172A), card surfaces (#1E293B), and vibrant indigo accents (#6366F1).
- **Modern Typography:** Clean Inter font hierarchy with high contrast and legible body text.
- **Vector Iconography:** Integrated Lucide Icons for intuitive visual elements (Calendar, MapPin, Tag, User, Search).

Component Library & Responsive Layout

- **Tailwind CSS v4:** High-performance utility-first styling with zero runtime CSS overhead.
- **Modular UI Components:** Reusable components: Navbar, EventCard, CategoryFilter, HeroBanner, CapacityBar.
- **Responsive Grid Layout:** Fluid layouts adapting smoothly from 4-columns on desktop to 1-column on mobile.
- **Micro-Interactions:** Smooth card hover lifts, animated tab pill indicators, and toast feedback alerts.

Slide 2 (M2): Multi-Category Discovery & Real-Time Filtering

Multi-Category Event Catalog

- **6 Event Categories:** Quick filter tabs: Movies, Stream, Events, Plays, Sports, and Activities.
- **Active Category Badges:** Highlighted pill tabs with animated active indicators for clear user context.
- **Rich Event Cards:** Displays event poster image, title, category, date badge, venue, and price.
- **Friendly Empty States:** Clean fallback illustrations and messages when no shows match the filter criteria.

Instant Search & URL Synchronization

- **Real-Time Search Bar:** Instant client-side debounced search matching title, venue location, and keywords.
- **Date Filter Picker:** Enables users to discover upcoming events happening today, this weekend, or next month.
- **URL Query Sync:** Search and category selections sync to URL (?category=sports&q=...) for bookmarking.
- **Framer Motion Transitions:** Smooth layout animations as event cards reorder and filter dynamically.

Slide 3 (M2): Organizer Dashboard & Attendee Ticket Portals

Organizer Management Dashboard

- **Live Metrics Cards:** Counters for Total Events Hosted, Tickets Sold, and Total Revenue generated.
- **Capacity Progress Bars:** Visual progress bar comparing tickets remaining vs total capacity with color thresholds.
- **List Your Show Modal:** Interactive modal form with validation for title, category, date-time, and poster URL.
- **Show Management Actions:** Quick action triggers to edit event details or delete expired listings.

Attendee "My Tickets" Portal

- **Booking History Cards:** Chronological timeline of all booked tickets with unique booking references.
- **Ticket Status Indicators:** Clear status badges: "Confirmed", "Upcoming", or "Completed".
- **Pass Download Triggers:** Single-click buttons to download official PDF tickets directly to device.
- **Gate QR Code Modal:** Popup modal displaying high-resolution scannable QR code for entry verification.

Slide 4 (M2): Frontend Challenges & Deliverables

UI Engineering Challenges Solved

- **Challenge: Hydration Mismatch:** React 19 date formatting discrepancies between server and client locales.
- **Solution Implemented:** Created a standardized formatDate() helper and wrapped localized UI in mount guards.
- **Challenge: Layout Shifts (CLS):** Layout jumping when external event poster images load asynchronously.
- **Solution Implemented:** Applied fixed aspect-ratio containers with shimmer skeleton loaders.

Key Frontend Deliverables

- **Semantic HTML5 Structure:** Strict semantic elements (<header>, <nav>, <main>, <article>) for accessibility.
- **Zustand State Store:** Lightweight global store for auth and cart states without unnecessary re-renders.
- **1-Click Demo Buttons:** Quick demo login shortcuts on sign-in page for instant live presentation review.
- **Cross-Device Usability:** Fully verified on mobile screens, laptops, and desktop browsers.

Slide 1 (M3): Relational Schema Design & Entity Relations

PostgreSQL Tables (src/db/schema.ts)

• **users Table:** id (PK), name, email (unique), passwordHash, role (attendee/organizer), createdAt.

• **events Table:** id (PK), title, description, category, location, startsAt, endsAt, ticketPriceCents, capacity, ticketsRemaining, organizerId (FK).

• **bookings Table:** id (PK), reference (unique hash), eventId (FK), userId (FK), stripeSessionId (unique), quantity, totalCents, status, createdAt.

Referential Integrity & Constraints

• **Foreign Key Relations:** events.organizerId -> users.id; bookings.eventId -> events.id; bookings.userId -> users.id.

• **Cascade Deletes:** onDelete: "cascade" ensures child events and bookings clean up if user is removed.

• **Unique Constraints:** Guarantees unique emails, booking reference hashes, and Stripe session IDs.

• **ACID Guarantees:** Full transactional consistency prevents corrupted booking or financial records.

Slide 2 (M3): Drizzle ORM Integration & Type Safety

Advantages of Drizzle ORM

- **Zero Runtime Overhead:** Lightweight TypeScript SQL dialect library without bulky engine binaries.
- **SQL-Like Syntax:** Clean and transparent queries: db.select().from(events).where(eq(events.id, id)).
- **Automatic Type Inference:** Generates TypeScript types (User, Event, Booking) directly from table definitions.
- **Serverless Optimization:** Minimal bundle size allows near-instant cold start times on cloud serverless.

Migrations & Connection Pooling

- **Drizzle Kit Push:** npm run db:push syncs schema changes directly to PostgreSQL in seconds.
- **Connection Pooling:** Configured native pg.Pool in src/db/index.ts to handle concurrent database requests.
- **SSL Compatibility:** Configured SSL connections for cloud-hosted Neon Serverless PostgreSQL.
- **Compile-Time Safety:** Catches column misspellings and type errors during build instead of runtime.

Slide 3 (M3): Inventory Concurrency & Data Integrity

Overselling Prevention & Concurrency

- **Real-Time Inventory State:** Every show tracks original capacity alongside dynamic ticketsRemaining counter.
- **Atomic Ticket Decrement:** Tickets decremented atomically in SQL upon confirmed checkout payment.
- **Conditional SQL Updates:** UPDATE events SET ticketsRemaining = ticketsRemaining - N WHERE id = X AND ticketsRemaining >= N.
- **Automatic Rollback:** If remaining tickets < requested quantity, 0 rows update and booking fails safely.

Indexing & Session Deduplication

- **Unique Indexing:** uniqueIndex("bookings_session_idx").on(table.stripeSessionId) in schema.
- **Idempotency Protection:** Prevents duplicate ticket allocation if user refreshes the success callback URL.
- **Cryptographic References:** Unique 12-char randomized booking reference prevents ticket forgery.
- **Foreign Key Indexing:** Indices ensure rapid query joins when retrieving user booking history.

Slide 4 (M3): Challenges Overcome & Database Seeding

Database Challenges Solved

- **Challenge: Price Precision:** Floating-point calculation rounding discrepancies in currencies.
- **Solution Implemented:** Stored all prices as integers in cents (ticketPriceCents, totalCents) as fintech standard.
- **Challenge: Timezones:** Timezone discrepancies between venue local time and UTC database timestamps.
- **Solution Implemented:** Stored timestamps in standard UTC and formatted to local timezone on the client.

Database Seeding & Deliverables

- **Automated Seed Script:** src/db/seed.ts restores complete presentation demo data with npm run db:seed.
- **Pre-Seeded Demo Accounts:** Pre-configures organizer@demo.com and attendee@demo.com for instant testing.
- **10+ Multi-Category Shows:** Populates realistic demo events across cinema, concerts, sports, and plays.
- **Zero Migration Drift:** Drizzle Kit keeps local development and production cloud schemas completely in sync.

Slide 1 (M4): Stripe Payment Gateway Integration

Stripe Checkout Lifecycle

- **Checkout Session Creation:** Server initiates stripe.checkout.sessions.create() with items, price, and currency.
- **Secure Hosted Checkout:** User redirected to official Stripe-hosted payment form; card details entered securely.
- **Success Callback Route:** Redirects to /checkout/success?session_id={CHECKOUT_SESSION_ID} on payment completion.
- **Cancellation Callback:** Redirects to /checkout/cancel if dismissed, releasing cart without phantom booking.

Server Verification & PCI Compliance

- **Direct Server Verification:** Server queries Stripe API directly to verify session.payment_status === "paid".
- **Full PCI-DSS Compliance:** Credit card numbers never touch our servers; 100% processed by Stripe infrastructure.
- **Test Mode Sandbox:** Configured STRIPE_SECRET_KEY in test mode using official test card numbers (4242...).
- **Session Metadata:** Passes eventId, userId, and ticket quantity in Stripe session metadata for reconciliation.

Slide 2 (M4): Programmatic PDF Ticket Generation (PDFKit)

PDFKit Vector Generation Engine

- **Server-Side Vector Engine:** PDFKit generates vector PDF documents entirely in Node.js server environment.
- **Official Ticket Layout:** Standard A4 format with purple "Evently E-Ticket" banner and booking reference.
- **Event Details Card:** Includes formatted event title, venue address, show start time, and ticket count.
- **Attendee Identity Block:** Prints attendee full name and registered email for identity validation at venue.

In-Memory Buffering & Binary Streaming

- **Embedded QR Code Buffer:** Embeds dynamically generated QR PNG buffer directly using doc.image(qrBuffer).
- **In-Memory Chunk Buffer:** Collects PDF chunks into Buffer.concat() with zero temporary files written to disk.
- **Instant Download Response:** Streams binary response with Content-Type: application/pdf for 1-click download.
- **Printable Pass Format:** Clean vector design suitable for both smartphone screens and paper printouts.

Slide 3 (M4): Scannable QR Codes & Email Delivery

QR Code Generation & Gate Security

- **QR Code Generator:** qrcode package in src/lib/qr.ts converts booking reference to high-res PNG buffers.
- **Cryptographic Reference:** Encodes unique 64-character booking reference hash for instantaneous lookup.
- **Gate Camera Validation:** Event staff can scan QR with mobile camera to instantly verify ticket legitimacy.
- **Anti-Counterfeiting:** Impossible to forge because reference must match an active confirmed database record.

Automated Email Confirmation (Nodemailer)

- **Nodemailer Pipeline:** Configured in src/lib/email.ts for automated transactional emails upon payment.
- **HTML Receipt Delivery:** Dispatches confirmation email with venue address, date, and booking reference.
- **Asynchronous Dispatch:** Dispatched in background so UI checkout success redirect remains instant.
- **Resilient Access:** If email delivery fails, tickets remain 100% accessible via My Tickets portal.

Slide 4 (M4): Testing, Cloud Deployment & Future Scope

Testing & Quality Assurance

- **Postman API Test Suites:** Constructed API test suites validating status codes (200, 400, 401, 403, 404).
- **Boundary Edge Cases:** Verified that zero-ticket bookings and unauthorized organizer actions are rejected.
- **Strict TypeScript 5.9:** npm run typecheck verifies zero runtime undefined or type mismatch errors.
- **ESLint 9 Code Hygiene:** Enforces clean code conventions, hook dependencies, and unused variable cleanup.

Cloud Deployment & Future Scope

- **Vercel Serverless Hosting:** Automated CI/CD deployment on every Git push; serverless API functions scale automatically.
- **Neon Serverless PostgreSQL:** Cloud PostgreSQL database with encrypted SSL connection pooling (sslmode=require).
- **Encrypted Secrets:** DATABASE_URL, JWT_SECRET, STRIPE_SECRET_KEY stored securely in Vercel settings.
- **Future Enhancements:** Interactive SVG seat selection matrix, organizer revenue analytics, and WhatsApp alerts.

TEAM RESPONSIBILITY & MODULE MATRIX

Role / Member	Tech Stack	Core Deliverables	Key Codebase Files
Member 1 **Backend Lead**	Next.js 16 App Router, JWT, bcryptjs	Full-stack architecture, REST API route hierarchy, RBAC authorization guards, httpOnly secure cookies	src/app/api/* src/lib/auth.ts src/lib/jwt.ts
Member 2 **Frontend Lead**	React 19, Tailwind CSS v4, Framer Motion, Lucide	6-category event discovery catalog, real-time search filters, organizer dashboard, attendee ticket portal	src/app/events/* src/components/* src/app/dashboard/*
Member 3 **Database Lead**	PostgreSQL, Drizzle ORM, Drizzle Kit	Relational schema design, cascade rules, atomic inventory decrements, concurrency control, database seeder	src/db/schema.ts src/db/index.ts src/db/seed.ts
Member 4 **Payment & DevOps**	Stripe API, PDFKit, QRCode, Vercel, Neon	Stripe checkout integration, programmatic PDF ticket generation, gate QR verification, Postman test suites, cloud deploy	src/lib/pdf.ts src/lib/qr.ts src/lib/stripe.ts